Código fonte e teste de unidade

Nodify — Sistema de Acessibilidade Cefálica para Spotify

Data: 09/10/2026 **Casos de uso:** UC01, UC02, UC03 **Plataforma:** Android · Kotlin 2.0.21 · Jetpack Compose

3.1 Código fonte

Desenvolvimento do código do sistema conforme as especificações e padrões de codificação definidos. Indicar padrões de qualidade de código e repositório do código-fonte.

Repositório do código-fonte

Endereço	`https://github.com/nicoleedias/acessibilidade-spotify`
Controle de versão	Git · branch principal `main` · 73 commits
Linguagem e plataforma	Kotlin 2.0.21 · Android Gradle Plugin 8.13.2 · JVM alvo 17
Níveis de API	`minSdk` 29 (Android 10) · `targetSdk` e `compileSdk` 35
Interface	Jetpack Compose (Material 3)
Segredos	O *client ID* do Spotify fica em `local.properties`, fora do versionamento. Nenhuma credencial é commitada.

Padrões de codificação adotados

Os padrões abaixo estão registrados no arquivo `CLAUDE.md`, na raiz do repositório, e valem para qualquer pessoa ou ferramenta que altere o código.

Aspecto	Padrão	Por que foi adotado
Arquitetura	MVVM em quatro camadas: apresentação, processamento, integração e dados.	A camada de domínio não tem nenhum *import* de Android, o que a torna testável sem emulador.
Guia de estilo	Kotlin Official Style Guide, verificado automaticamente pelo ktlint.	Indentação, ordem de imports, nomes e formatação deixam de ser discussão de revisão.
Imutabilidade	`val` sobre `var`; `data class` para estado; *sealed class* para máquinas de estado.	Estado de tela imutável elimina uma classe inteira de defeito de concorrência.
Nulabilidade	Operador `!!` proibido. Usa-se `requireNotNull()` com mensagem ou trata-se o ramo nulo.	Falha com diagnóstico em vez de *NullPointerException* anônima.
Nomenclatura	Identificadores em inglês; textos de interface em `res/values/strings.xml`.	Separa código de conteúdo e deixa o app pronto para tradução.
Compose	Um Composable por tela; estado elevado (*state hoisting*); sem regra de negócio no Composable.	É o que permite testar as telas sem subir injeção de dependência.
Comentários	Só quando o *porquê* não é óbvio. Proibido comentário descritivo, código comentado e TODO.	Comentário que repete o código envelhece e passa a mentir.
Commits	*Conventional Commits* com o caso de uso entre colchetes — `feat(calibration): … [UC02]`.	Liga cada commit ao requisito que ele atende, de forma rastreável.
Persistência	Migração de banco é sempre arquivo novo; migração existente nunca é editada.	Editar migração aplicada corrompe a base de quem já atualizou.

Ferramentas de garantia de qualidade

Ferramenta	Versão	Verifica	Comando	Observação
ktlint	12.1.1	Formatação e estilo	`./gradlew ktlintCheck`	Modo Android ativo; `ktlintFormat` corrige automaticamente.
detekt	1.23.7	Análise estática	`./gradlew detekt`	Configuração própria em `config/detekt.yml`; complexidade, tamanho de função e code smells.
Kover	0.9.1	Cobertura de testes	`./gradlew koverHtmlReportDebug`	Exclui código gerado (Hilt, Room, Compose), que inflaria o indicador.
JUnit 4 + MockK	4.13.2 / 1.13.13	Testes unitários	`./gradlew test`	MockK simula classes `final` e funções `suspend` sem configuração extra.

Portão de qualidade obrigatório

Nenhuma alteração é integrada sem que o comando abaixo passe por completo. Os três verificadores são executados em sequência e qualquer falha interrompe a entrega.

```
./gradlew ktlintCheck detekt test
BUILD SUCCESSFUL — 186 testes, 0 falhas
```

> Os padrões não dependem de disciplina individual: estilo e complexidade são verificados por ferramenta, e o portão roda antes de qualquer integração. O que o revisor humano examina é o que a ferramenta não alcança — aderência ao requisito, segurança e preservação do comportamento existente.

3.2 Teste de unidade

Testes realizados pelos desenvolvedores para verificar se cada unidade do código funciona corretamente de forma isolada. Apresentar o resultado dos indicadores de qualidade do código (% de cobertura; taxa de sucesso).

A suíte unitária tem **186 testes** que rodam na JVM, sem emulador nem aparelho, em menos de quatro segundos. Cada teste exercita uma classe isolada, com as dependências substituídas por simulações (MockK). São executados a cada alteração, como parte do portão de qualidade.

Indicadores de qualidade

Indicador	Resultado
Quantidade de testes unitários	**186**
Testes aprovados	**186**
Testes reprovados	**0**
Taxa de sucesso	**100 %**
Cobertura de linhas — núcleo lógico	**71,4 %** (543/760)
Cobertura de linhas — todo o aplicativo	**22,0 %** (543/2.463)
Cobertura de ramos (*branches*)	26,2 % (208/793)
Cobertura de classes	44,7 % (68/152)
Tempo de execução da suíte	3,6 s

> **Por que dois números de cobertura.** O indicador global (22,0 %) considera todas as 2.463 linhas do aplicativo, inclusive as 1.300 linhas de *Composables* — a descrição declarativa das telas — e as declarações de tema, navegação e injeção de dependência, que não contêm lógica e não são alcançáveis por teste de unidade. O indicador do **núcleo lógico (71,4 %)** considera apenas o código com comportamento verificável sem dispositivo. Os dois estão apresentados porque o primeiro sozinho subestima a qualidade da suíte, e o segundo sozinho esconderia o que ainda não é testado.

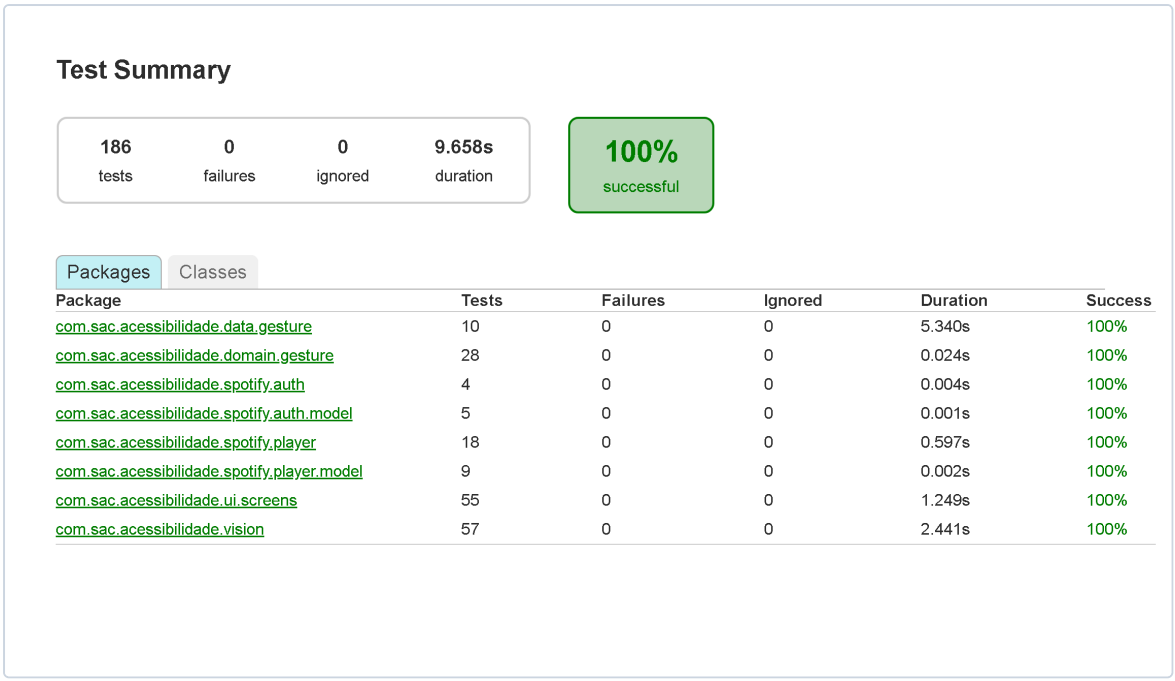
Cobertura por camada

Camada	Linhas	Cobertura	Leitura do resultado
Domínio — regras de gesto e calibração	69/69	**100 %**	Kotlin puro, sem Android. Alvo declarado de cobertura total.
Dados — mapeamento gesto→ação	32/32	**100 %**	Lógica do repositório com acesso a dados simulado.
Integração Spotify — comandos e faixa	38/38	**100 %**	Inclui a tradução de 403 e 404 em mensagem para o usuário.
Modelos da API Spotify	21/31	67,7 %	O não coberto são campos de desserialização sem lógica.
Visão computacional	168/310	54,2 %	As classes puras estão em 100 %; o restante é MediaPipe, que exige câmera.
Apresentação — ViewModels e telas	204/1.513	13,5 %	Os 4 ViewModels estão em 100 %; as 1.300 linhas restantes são Composables.
Autenticação OAuth	6/132	4,5 %	Depende do Android Keystore; coberto por teste instrumentado.
Tema, navegação e injeção de dependência	0/249	0 %	Declarações sem lógica — não há comportamento a verificar.

As camadas que concentram a regra de negócio — domínio, dados e integração com o Spotify — estão em **100 %**. Esse é o resultado da decisão de arquitetura de manter o domínio sem dependência de Android.

Classes com 100 % de cobertura de linhas

Camada	Classes
Domínio	CalibrationThresholdCalculator, CalibrationThresholds, Gesture, GestureDisplayNames, SpotifyAction
Dados	GestureMappingRepositoryImpl, GestureMappingEntity
Spotify	SpotifyCommandRepositoryImpl, SpotifyPlayerRepositoryImpl, PkceUtils, SpotifyTokens, AlbumItem, TrackItem
Visão	GestureClassifier, HeadGestureStabilizer, CaptureQualityEvaluator, CaptureIssue
Apresentação	LoginViewModel, HomeViewModel, GestureConfigViewModel, PlayerAtivoViewModel



Test Summary

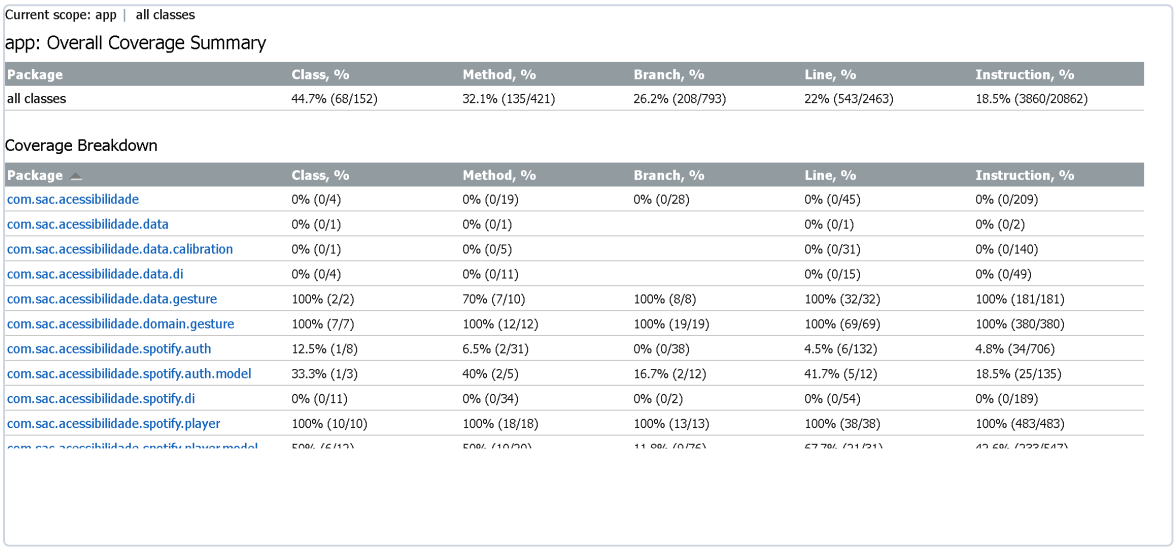
186	0	0	9.658s
tests	failures	ignored	duration

100% successful

Packages | Classes

Package	Tests	Failures	Ignored	Duration	Success
com.sac.acessibilidade.data.gesture	10	0	0	5.340s	100%
com.sac.acessibilidade.domain.gesture	28	0	0	0.024s	100%
com.sac.acessibilidade.spotify.auth	4	0	0	0.004s	100%
com.sac.acessibilidade.spotify.auth.model	5	0	0	0.001s	100%
com.sac.acessibilidade.spotify.player	18	0	0	0.597s	100%
com.sac.acessibilidade.spotify.player.model	9	0	0	0.002s	100%
com.sac.acessibilidade.ui.screens	55	0	0	1.249s	100%
com.sac.acessibilidade.vision	57	0	0	2.441s	100%

Figura 1 — Relatório do Gradle: 186 testes, 0 falhas, 100 % de sucesso.



Current scope: app | all classes

app: Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
all classes	44.7% (68/152)	32.1% (135/421)	26.2% (208/793)	22% (543/2463)	18.5% (3860/20862)

Coverage Breakdown

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
com.sac.acessibilidade	0% (0/4)	0% (0/19)	0% (0/28)	0% (0/45)	0% (0/209)
com.sac.acessibilidade.data	0% (0/1)	0% (0/1)		0% (0/1)	0% (0/2)
com.sac.acessibilidade.data.calibration	0% (0/1)	0% (0/5)		0% (0/31)	0% (0/140)
com.sac.acessibilidade.data.di	0% (0/4)	0% (0/11)		0% (0/15)	0% (0/49)
com.sac.acessibilidade.data.gesture	100% (2/2)	70% (7/10)	100% (8/8)	100% (32/32)	100% (181/181)
com.sac.acessibilidade.domain.gesture	100% (7/7)	100% (12/12)	100% (19/19)	100% (69/69)	100% (380/380)
com.sac.acessibilidade.spotify.auth	12.5% (1/8)	6.5% (2/31)	0% (0/38)	4.5% (6/132)	4.8% (34/706)
com.sac.acessibilidade.spotify.auth.model	33.3% (1/3)	40% (2/5)	16.7% (2/12)	41.7% (5/12)	18.5% (25/135)
com.sac.acessibilidade.spotify.di	0% (0/11)	0% (0/34)	0% (0/2)	0% (0/54)	0% (0/189)
com.sac.acessibilidade.spotify.player	100% (10/10)	100% (18/18)	100% (13/13)	100% (38/38)	100% (483/483)

Figura 2 — Relatório de cobertura (Kover), com o detalhamento por pacote.

Como reproduzir os indicadores

```
./gradlew test                    # taxa de sucesso
./gradlew koverHtmlReportDebug    # cobertura
```

Os relatórios são gerados em `app/build/reports/tests/testDebugUnitTest/index.html` e `app/build/reports/kover/htmlDebug/index.html`.

> **Limite declarado.** O projeto tem ainda 84 testes instrumentados (33 de integração e 51 funcionais) que compilam, mas exigem aparelho ou emulador. Não entram na demonstração nem nos indicadores acima — nenhum resultado foi presumido.

Conjunto de demonstração

Para execução ao vivo foi preparado um subconjunto de **53 testes** em quatro classes, cobrindo os três casos de uso. São testes de lógica pura: não usam corrotinas, rede, banco nem dispositivo, o que os torna rápidos e

determinísticos.

```
./gradlew testDebugUnitTest -Pdemo
CalibrationThresholdCalculatorTest > aplica 75 por cento do pico PASSED  (… 53 linhas)
BUILD SUCCESSFUL in 7s — 53 testes, 0 falhas
```

O conjunto ignora o cache do Gradle de propósito: os testes são de fato executados a cada chamada e listados um a um, em vez de o build reportar `FROM-CACHE` sem mostrar nada.

Classe de teste	Testes	Caso de uso	O que demonstra
CalibrationThresholdCalculatorTest	11	UC02	Limiar em 75 % do pico medido, mínimos de segurança por eixo e polaridade aprendida da câmera.
CaptureQualityEvaluatorTest	18	UC02	Luz, enquadramento e número de rostos; iluminação tem prioridade sobre “rosto não detectado”.
GestureClassifierTest	15	UC01	Cada gesto do vocabulário é classificado; piscada tem prioridade sobre pose.
GestureDisplayNamesTest	9	UC03	Ida e volta entre rótulo e ação, e unicidade dos rótulos.